

1. Implementa o algoritmo Token Bucket para limitar taxa de requisições, permitindo consumo até a capacidade disponível.
2. O bucket começa cheio; cada chamada a `consume(requested)` tenta retirar a quantidade pedida de tokens.
3. Antes do consumo, `refill()` calcula o tempo decorrido via `System.nanoTime()` e repõe tokens proporcionalmente à taxa `refillRate`.
4. A reposição usa `Math.min(capacity, ...)` para nunca ultrapassar a capacidade máxima configurada.
5. Se houver tokens suficientes, subtrai `requested` e retorna `true`; caso contrário retorna `false`.
6. Usa `synchronized` em `consume` para exclusão mútua, evitando corrida entre threads concorrentes.
7. Conceitos principais: encapsulamento, sincronização, controle de concorrência e medição de tempo em nanossegundos.
8. Complexidade por operação: $O(1)$ tempo e $O(1)$ espaço, pois usa apenas cálculos simples e poucos campos.
9. O main demonstra seis requisições de 3 tokens com pausas de 500 ms, mostrando recarga gradual.
10. Limitação: precisão depende do relógio/scheduler e somente `consume` é sincronizado; alta contenção pode reduzir desempenho.